c10vis-poem / GenieX

Code | Pull requests | Agents | Actions | Projects | Wiki | Security and quality | Insights | Settings

Watch 0 | Fork 0

Run frontier LLMs and VLMs locally on Qualcomm devices across NPU, GPU, and CPU with a few lines of code

BSD 3-Clause "New" or "Revised" License

geniex.aihub.qualcomm.com/en/get-started/what-is-geniex

Code of conduct

Contributing

Security policy

0 stars | 0 forks | 0 watching | 1 branch | 0 tags | Activity

Public repository · Forked from qualcomm/GenieX

1 Branch | 0 Tags | Go to file | Go to file | Add file | Code

This branch is 84 commits behind qualcomm/GenieX:main . | Contribute | Sync fork

mengshengwu Merge pull request qualcomm#1165 from ericcurtin/feat/docker-hub-mode...

6285427 · 2 weeks ago

.claude	chore(dx): document lcov 1.x/msys2 quirk...	2 months ago
.github	ci(build-sdk): override cached HTP cert pa...	2 weeks ago
bindings	fix(sdk): route Docker Hub prefix pulls pas...	2 weeks ago
cli	fix(sdk): route Docker Hub prefix pulls pas...	2 weeks ago
docs	Merge pull request qualcomm#1160 from ...	3 weeks ago
examples/python	chore(android): drop demo app, moved to ...	last month
notes	ci(release): publish stable tags to producti...	28 days ago
scripts	chore(sdk): drop leftover nexa-sdk referen...	last month
sdk	fix(sdk): route Docker Hub prefix pulls pas...	2 weeks ago
tests	fix(python): fix VLM multi-turn image guar...	27 days ago
third-party	Trim comments and update geniex-qairt s...	3 weeks ago
.bazelignore	chore(build): bazel-ignore .claude worktre...	2 months ago
.bazelrc	chore(dx): add coverage skill and gate rele...	2 months ago
.bazelversion	chore: lock bazel version	3 months ago
.clang-format	chore: add .clang-format configuration	3 months ago
.gitattributes	feat(examples): add RAG-LLM	3 months ago

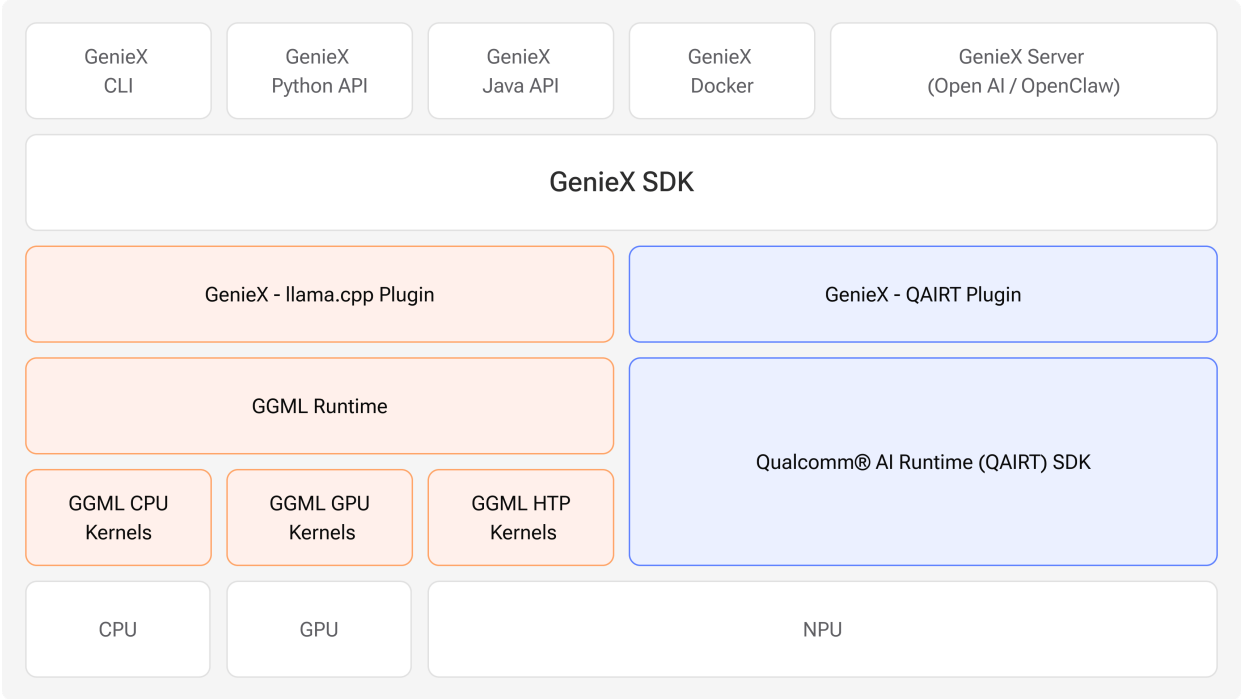
.gitignore	ci: run model-running pytest cells on QDC …	last month
.gitmodules	chore(build): track public geniex-qairt repo…	last month
.mailmap	chore: add .mailmap to unify author email …	last month
BUILD.bazel	chore(cli): use bazel to generate protobuf …	3 months ago
CLAUDE.md	docs: standardize on runtime / compute u…	2 months ago
CODE-OF-CONDUCT.md	chore: add CODE-OF-CONDUCT.md (Contri…	last month
CODEOWNERS	chore: update CODEOWNERS with all cont…	last month
CONTRIBUTING.md	docs(contributing): switch merge strategy …	2 months ago
GenieX-Logo-Hor-1-Black.png	docs: sync scripts/README.md from upst…	last month
GenieX-Logo-Hor-1-White.png	docs: sync scripts/README.md from upst…	last month
LICENSE	docs: relicense from Apache 2.0 to BSD 3-…	last month
MODULE.bazel	fix(bazel): update go bindings use_repo na…	last month
MODULE.bazel.lock	chore(build): regenerate MODULE.bazel.lo…	2 months ago
NOTICE	docs: fix stale nlohmann/json license path…	last month
README.md	feat(sdk): pull models from Docker Hub's …	3 weeks ago
README_zh-CN.md	docs: add Simplified Chinese README an…	3 weeks ago
SECURITY.md	feat: rename `qualcomm/geniex` and update…	last month
repolint.json	fix(ci): unify source license headers to BS…	27 days ago

README | Code of conduct | Contributing | License | Security

The easiest way to run frontier LLMs & VLMs locally on Qualcomm devices

Documentation · Quickstart · Models · Community

GenieX is an **on-device Gen AI inference runtime for Qualcomm devices**. Bring almost any GGUF model from Hugging Face — or a pre-compiled bundle from Qualcomm AI Hub — and run it locally on the **Hexagon NPU, Adreno GPU, or CPU** in a few lines of code. One C SDK underneath, exposed through a CLI, Python, Kotlin/Java, Docker, and an OpenAI-compatible server. It is the community version of Qualcomm GENIE.



Supported platforms

GenieX runs **only on Qualcomm Snapdragon**. Find your platform, then jump straight to the interface you want to use.

Platform	Example devices	Jump to a quickstart
🪟 **Windows ARM64** *(Compute)*	Snapdragon X · X Elite	CLI · Python · Local server
🤖 **Android** *(Mobile)*	Snapdragon 8 Elite · 8 Elite Gen 5	Android SDK
🐧 **Linux ARM64** *(IoT)*	Dragonwing QCS9075	CLI · Docker · Python

> No device on hand? Spin up a remote session on Qualcomm Device Cloud.

Quickstart

Pick your interface below. Each one follows the same three steps — **Install**, **Run**, and **Docs** — and shows both runtimes: a **GGUF** model from Hugging Face (`llama_cpp`) and a **pre-compiled bundle** from Qualcomm AI Hub (`qairt` , NPU).

CLI

Windows ARM64 Linux ARM64

Install

- **Windows ARM64** — download the installer, run it, then open a new terminal.
- **Linux ARM64** — one line, no `sudo` :

```
curl -fsSL https://qaihub-public-assets.s3.us-west-2.amazonaws.com/qai-hub-geniex/install.sh | sh
```

Run — chat with any model in one line (drag in an image for VLMs):

```
# GGUF from Hugging Face → llama.cpp (NPU / GPU / CPU)
geniex infer google/gemma-4-E4B-it-qat-q4_0-gguf
```

```
# Pre-compiled bundle from Qualcomm AI Hub → Qualcomm AI Engine Direct (NPU)
geniex infer ai-hub-models/Qwen2.5-VL-7B-Instruct

# GGUF from Docker Hub (https://hub.docker.com/u/ai) → llama.cpp (NPU / GPU / CPU)
geniex infer docker.io/ai/gemma3
```

📖 **Docs** — Install · Quickstart · Command reference

Python

Windows ARM64 Linux ARM64

Install

```
pip install geniex
```

Run — mirrors Hugging Face `transformers` (`from_pretrained()` → `.generate()`):

```
# GGUF from Hugging Face → llama.cpp
from geniex import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("unsloth/Qwen3.5-2B-GGUF", precision="Q4_0")

messages = [{"role": "user", "content": "What is 2+2?"}]
prompt = model.tokenizer.apply_chat_template(messages, add_generation_prompt=True)

for chunk in model.generate(prompt, max_new_tokens=256, stream=True):
    print(chunk, end="", flush=True)

model.close()
```

```
# Pre-compiled bundle from Qualcomm AI Hub → Qualcomm AI Engine Direct (NPU)
from geniex import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("ai-hub-models/Qwen3-4B")

messages = [{"role": "user", "content": "What is 2+2?"}]
prompt = model.tokenizer.apply_chat_template(messages, add_generation_prompt=True)

for chunk in model.generate(prompt, max_new_tokens=256, stream=True):
    print(chunk, end="", flush=True)

model.close()
```

📖 **Docs** — Install · Quickstart · API reference

OpenAI-compatible server

Windows ARM64 Linux ARM64

Install — ships with the CLI (install above).

Run — pull any model (GGUF or Qualcomm AI Hub bundle), then serve an OpenAI-compatible API:

```
geniex pull ai-hub-models/Qwen3-4B-Instruct-2507
geniex serve   # serves http://127.0.0.1:18181/v1
```

```
curl http://127.0.0.1:18181/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "ai-hub-models/Qwen3-4B-Instruct-2507",
    "messages": [{"role": "user", "content": "Hello!"}]
  }'
```

Point any OpenAI client at `http://127.0.0.1:18181/v1` — no code changes.

📖 **Docs** — Local server guide

Android (Kotlin / Java)

Android

Install — add the SDK to your app module's `build.gradle.kts`:

```
dependencies {
    implementation("com.qualcomm.qti:geniex-android:0.3.1")
}
```

Run — fastest path is the sample app (chat UI, model picker for GGUF + Qualcomm AI Hub bundles, VLM support):

The Android demo app lives in `qualcomm/ai-hub-apps`. Clone it, open the sample app in Android Studio, and hit **Run**.

📖 **Docs** — Install · Quickstart · API reference

Docker

Linux ARM64

Install

```
docker pull docker.io/qualcomm/geniex:latest
```

Run — the container wraps the CLI, so `geniex infer …` works exactly as above.

📖 **Docs** — Docker guide

C / C++ SDK

Windows ARM64 Linux ARM64 Android

Install — link against the single C header `sdk/include/geniex.h`; every other interface is a thin wrapper over it.

📖 **Docs** — sdk/README.md · notes/build.md

Models

GenieX has two runtimes so you get **broad model coverage** *and* **peak Snapdragon performance** in one stack. Both LLMs and VLMs are supported.

	llama.cpp (`llama_cpp`)	**Qualcomm AI Engine Direct** (`qairt`)
Get models from	Hugging Face (any GGUF)	Qualcomm AI Hub (pre-compiled)

	llama.cpp (`llama_cpp`)	**Qualcomm AI Engine Direct** (`qairt`)
Format	GGUF	Per-chipset bundle
Compute units	NPU · GPU · CPU	NPU only
Best for	Bringing your own GGUF	Highest NPU performance

> For llama.cpp, pick the `Q4_0` precision when prompted — it has the best Hexagon NPU support. See the Models guide → for the full list, precisions, and how to run a local model.

🤝 Contributing

Contributions are welcome! Before opening a PR, please read **CONTRIBUTING.md** for branch naming, commit / PR title format, pre-commit checks, and the FFI-update rule for public SDK headers.

🏗️ **Build** the CLI, SDK, or Python bindings	notes/build.md
▶️ **Run** & select compute units / pull models	notes/run.md
🏷️ **Release** — SemVer tags, channels, HTP signing	notes/release.md
📚 **All developer docs**	docs/README.md

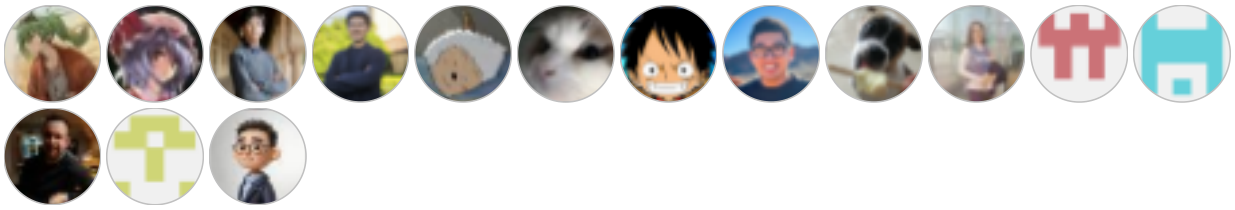
💬 Community & Contact

Questions, ideas, or want to show off what you built? Come say hi.

- 💬 **Slack** — ask questions and chat with the community in real time.
- 🐛 **GitHub Issues** — report a bug or request a feature.
- 🔗 **LinkedIn** — follow Qualcomm AI Hub for news and updates.

Contributors

Thanks to everyone building GenieX 💙



📄 License

BSD 3-Clause — see LICENSE and NOTICE.

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors

No contributors

Languages

Rust 27.3% **C++** 20.6% **Python** 18.6% **Go** 16.1% **C** 8.7% **Kotlin** 2.5% **Other** 6.2%

Suggested workflows

Based on your tech stack

Go
Build a Go project.
By GitHub Actions
Configure

Publish Python Package
Publish a Python Package to PyPI on release.
By GitHub Actions
Configure

SLSA Generic generator
Generate SLSA3 provenance for your existing release workflows
By Open Source Security Foundation (OpenSSF)
Configure

More workflows